

TOULOUSE INP - ENSEEIHT

2^E ANNÉE SN - IMAGE ET MULTIMÉDIA

Rapport de Projet : Traitement des Données Audiovisuelles



Julian Puech

Tous les codes et résultats détaillés dans ce projet
sont disponibles sur le dépôt GitHub suivant :

github.com/jlnpch/TAV

Année Universitaire 2025-2026

Table des matières

1	Introduction Générale	2
2	TP5 – Restauration d’images : Inpainting par Rapiécage	3
2.1	Contexte du TP	3
2.2	Inpainting par rapiécage	3
2.3	Optimisation de la méthode : avec l’algorithme de Criminisi	4
2.3.1	Implémentation de l’ordre de priorité	4
2.3.2	Problème de tiling	4
2.3.3	Améliorations et paramétrages intermédiaires	4
2.3.4	Amélioration finale : ajout d’aléatoire dans l’algorithme)	6
2.4	Conclusion : problème de coutures	6
3	TP7 – Photomontage et Equation de Poisson	7
3.1	Contexte du TP	7
3.2	Utilisation de l’équation de Poisson pour un effet de gravure	7
3.2.1	Principe Mathématique de la Gravure	7
3.2.2	Flou gaussien et génération du relief	7
3.2.3	Résultat final de l’effet de gravure	8
3.3	Conclusion	8
4	Experimentation Mix TP5 et TP7	9
4.1	Concept et limite de l’approche locale	9
4.2	Concept et limite de l’approche en deux passes	9
5	TP10 – Experimentations avec des Effets Sonores	11
5.1	Contexte du TP	11
5.2	Spectral Gate	11
5.3	Modification de la phase d’un son	11
5.4	Effet de reverb	12
5.4.1	Première implémentation	12
5.4.2	Amélioration avec pre-delay et phase	13
5.4.3	Amélioration avec flou temporel	14
5.5	Conclusion sur les effets sonores	14
6	TP12 – Séparation de Sources	15
6.1	Contexte du TP	15
6.2	Apprentissage d’un réseau de type U-Net	15
6.3	Optimisations du modèle	15
7	Conclusion Générale	17

1 Introduction Générale

Ce projet a pour objectif d'approfondir les notions abordées en travaux pratiques de Traitement des Données Audiovisuelles (TAV). Pour ce faire, nous avons mis en œuvre et analysé plusieurs méthodes avancées en traitement d'image et de signal à travers cinq axes principaux :

- **TP5 - Inpainting par Rapièçage** : Restauration de zones manquantes dans une image par remplissage par patches via l'algorithme de Criminisi.
- **TP7 - Photomontage et Equation de Poisson** : Utilisation de l'équation de Poisson pour éditer les gradients d'une image et créer un effet "gravure".
- **TP5 et TP7** : Tentative d'amélioration de l'inpainting par patches à l'aide des gradients de l'image source.
- **TP10 - Effets sonores** : Manipulation de signaux audio dans le domaine fréquentiel à l'aide de la Transformée de Fourier à Court Terme (TFCT) pour créer différents effets sonores (Spectral Gate, modification de phase, Reverb...).
- **TP12 - Séparation de sources par Deep Learning** : Séparation de sources audio (isolation de la voix et de l'accompagnement) par l'apprentissage d'un réseau de neurones (U-Net).

Les TPs 5,7 et 10 sont traités entièrement en MATLAB tandis que le TP 12 est un notebook python utilisant Keras et Tensorflow pour l'entraînement d'un réseau de neurones. Tout les codes réalisés et utilisés pour ce projet sont dans le dossier /codes sur le dépôt GitHub (lien en couverture) tandis que les résultats produits par ces codes (sons et images) sont dans le dossier /resultats.

2 TP5 – Restauration d’images : Inpainting par Rapièçage

2.1 Contexte du TP

Comme constaté dans l’exercices 2 de ce TP, les méthodes par approche variationnelle comme l’inpainting par diffusion peuvent reconstituer des zones de faibles épaisseurs manquantes sur une image tout en préservant les arrêtes et contours de celle-ci. En revanche, elles ne permettent pas de combler de manière satisfaisante des trous plus grands, et ne peuvent donc pas être utilisées pour des problèmes de réalité diminuée qui consistent à supprimer entièrement un ou plusieurs objets présents sur une image. Pour pallier à ce problème une des méthodes possibles est l’inpainting par rapièçage. Cette méthode consiste à rechercher des patches de pixels dans les zones intactes de l’image, puis à les copier de manière itérative pour combler la région manquante. Cette approche parvient à reconstruire simultanément les structures géométriques et les textures complexes, rendant ainsi la disparition d’objets volumineux visuellement plus crédible.



FIGURE 1 – Image originale



FIGURE 2 – Résultat après inpainting par patches

2.2 Inpainting par rapièçage

Si la méthode d’inpainting présentée en TP fonctionne assez bien sur cet exemple, elle est selon le contexte bien moins performante quand par exemple la zone à effacer contient des objets avec des contours nets (ici en l’occurrence des arbres). On constate sur cet exemple que l’image obtenue après le rapièçage par défaut est peu convaincante avec des troncs d’arbres coupés et des frontières visibles entre les patches.

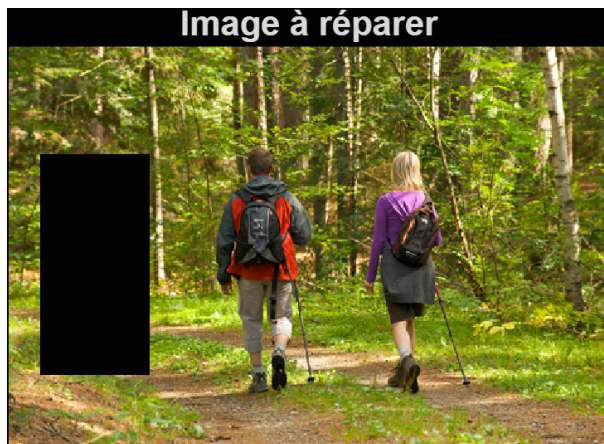


FIGURE 3 – Image avec région manquante



FIGURE 4 – Résultat en appliquant la méthode par défaut

2.3 Optimisation de la méthode : avec l'algorithme de Criminisi

2.3.1 Implémentation de l'ordre de priorité

Ce problème a également été constaté par Criminisi, Pérez et Toyama en 2003 qui ont pour améliorer les résultats mis en place un ordre de priorité dans le choix de la zone à compléter.

Le pixel p de la frontière à compléter est choisi de manière à maximiser la priorité $P(p) = C(p) \times D(p)$, où $C(p)$ représente le terme de confiance qui mesure la proportion de pixels intacts ou déjà reconstruits présents dans son voisinage et $D(p)$ le terme d'attache aux données. Dans notre implémentation, ce terme de données a été approximé par la variance locale du patch. ce qui permet de quantifier efficacement les contours des formes présentes sur le patch : un patch traversé par un contour net aura une variance élevée, ce qui augmentera certainement sa priorité de traitement. Cette stratégie assure que les arêtes ayant des contours soit prolongées en premier à l'intérieur de la région manquante, préservant ainsi la cohérence globale de l'image avant le remplissage des textures plus uniformes.

2.3.2 Problème de tiling

On constate sur cet exemple ci-dessous un premier problème, l'algorithme fait apparaître un motif de grille régulier très déplaisant pour l'oeil humain. Ceci n'apparaissait pas avec la première méthode qui était en partie stochastique.



FIGURE 5 – Image "reconstruite" avec ordre de priorité

Ce problème vient sûrement du fait que le terme de variance $D(p)$ écrase $C(p)$, ce qui force l'algorithme à sélectionner pour le pixel suivant celui qui suit le patch carré venant d'être ajouté et sélectionne donc le même patch pour remplir cette zone.

2.3.3 Améliorations et paramétrages intermédiaires

Une première tentative a été d'augmenter T la zone de recherche du meilleur patch en la passant de 50 à 100 pixels de largeur, le résultat semble meilleure car il offre plus de choix pour la sélection du patch mais le problème de grille/tiling est toujours bien visible :



FIGURE 6 – Image reconstruite avec T augmenté

Un autre paramètre pouvant être modifié est t la taille des patchs eux-même, en la réduisant, l'effet de grille est moins visible mais on constate toujours par endroits des répétitions de patchs. De plus en réduisant trop la taille des patchs, on les empêche de bien représenter des éléments macro-structuels de l'image (feuilles, brins d'herbes, branches...) ce qui ne permet pas de bien reconstituer le contexte et peut donner lieux à des résultats plus brouillons.



FIGURE 7 – Image reconstruite avec t réduit

Pour essayer de réduire l'influence la variance dans le choix des patchs, on peut également essayer de réduire le terme $D(p)$ en par exemple $D(p) = \log(1 + \text{var})$ pour donner moins d'importance aux contours et arrêter de les prolonger à l'infini. Cette modification semble améliorer légèrement la situation mais la variance prend toujours le dessus et l'effet de répétition est toujours présent. On a constaté que même en réduisant encore plus $D(p)$ en le ramenant sur l'intervalle $[0,1]$ (même ordre de grandeur que $C(p)$), le tiling est toujours en partie visible, cela vient en fait de la fonction $d_{\min}$ qui choisit pour un pixel p dont le voisinage est à remplir le pixel q dont le voisinage est le plus ressemblant à celui de p . Comme cette fonction choisit toujours le patchs de distance minimum, elle choisit pour deux pixel p et p' de voisinage similaires le même patch q d'où cet effet de répétition.



FIGURE 8 – Image reconstruite avec réduction de $D(p)$

2.3.4 Amélioration finale : ajout d'aléatoire dans l'algorithme)

Une solution possible est donc d'ajouter de l'aléatoire dans notre fonction d_{min} , pour cela on choisit pour un pixel p non pas le pixel q le plus proche, mais un des 5 plus proches tiré de manière aléatoire dans la liste. Cette méthode permet d'introduire du hasard dans notre algorithme et de retrouver des résultats sans effet de grille comme on avait pour notre solution de base qui était elle aussi partiellement aléatoire. Le résultat obtenu est nettement mieux, mais on constate toujours que le résultat n'est pas parfait car les coutures entre les patches sont toujours visibles et le résultat n'est pas sémantiquement correct puisqu'on observe un chemin qui "grimpe" sur les arbres.

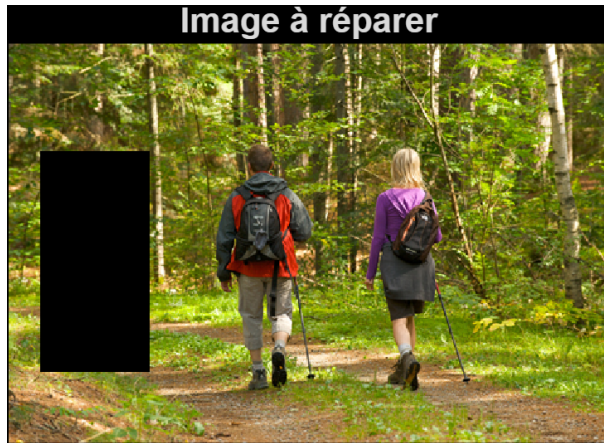


FIGURE 9 – Image avec région manquante



FIGURE 10 – Résultat avec d_{min} stochastique

2.4 Conclusion : problème de coutures

En conclusion nous avons pu constater que l'inpainting par patches est une méthode efficace pour reconstruire la structure et les textures d'une zone manquante. Toutefois, le fait de coller de manière directe et rigide les patches sur l'image entraîne de manière inévitable des démarcations/coutures nettes entre ces patches appelées hard seams.

3 TP7 – Photomontage et Equation de Poisson

3.1 Contexte du TP

Comme nous l'avons soulevé en conclusion du TP5, les méthodes de copie directe des intensités d'une image source vers une image cible comme l'inpainting par patches présentent un défaut récurrent qui est l'apparition de coutures visuellement évidentes. Ce phénomène s'explique par le fait que notre système visuel est beaucoup plus sensible aux brusques variations de contraste local (donc variation des gradients) qu'à l'intensité lumineuse globale d'une zone.

Une idée intéressante pour remédier à cela est de remplacer le collage brut des couleurs par le remplacement des gradients. On obtient finalement ces couleurs via l'intégration des gradients ce qui permet de conserver la structure de l'image.

Ce problème revient mathématiquement à résoudre l'équation de Poisson :

$$\Delta u = \nabla(\mathbf{v}) \quad (1)$$

où u est l'image inconnue à reconstruire et $\nabla(\mathbf{v})$ les gradients de l'image source.

Dans le TP7, nous avons utilisé cette méthode pour réaliser la fonction collage qui permet d'ajouter un objet dans une image cible de manière fluide.

3.2 Utilisation de l'équation de Poisson pour un effet de gravure

3.2.1 Principe Mathématique de la Gravure

Cette même équation qui nous a servis pour faire du photomontage dans le TP7, peut être modifiée pour créer divers effets comme présentés dans le papier de Pérez, Gangnet et Blake datant de 2003. Pour ce projet nous allons essayer de créer un effet de "gravure" qui permette de sculpter un texte (image source) dans un fond texturé mais régulier (image cible). Pour cela nous changeons l'équation de base en :

$$\Delta u = \nabla u + \alpha \nabla v. \quad (2)$$

Avec un α négatif, on soustrait les gradients de v aux gradients de u , ce qui permet de modifier localement le sens des transitions de lumière. Et donc d'augmenter ou réduire les lumières/ombres pour créer cet effet de gravure dont la profondeur est contrôlée par α .

3.2.2 Flou gaussien et génération du relief

Si on applique directement cette formule à un texte simple blanc sur noir, l'image obtenue n'est pas réaliste car les gradients de l'image source sont trop nets et artificiels. Pour un effet de profondeur, il faut une pente douce et une lumière directionnelle. On ajoute pour cela plusieurs étapes de traitements à notre image source avant de l'insérer dans l'équation : d'abord l'ajout d'un flou gaussien pour lisser les contours et créer une démarcation moins nette entre la partie gravée et celle non gravée, puis l'application d'un filtre directionnel de type $([-1 \ -1 \ 0; -1 \ 0 \ 1; 0 \ 1 \ 1])$ pour venir ajouter du relief avec un effet d'ombres et de lumières à la gravure :

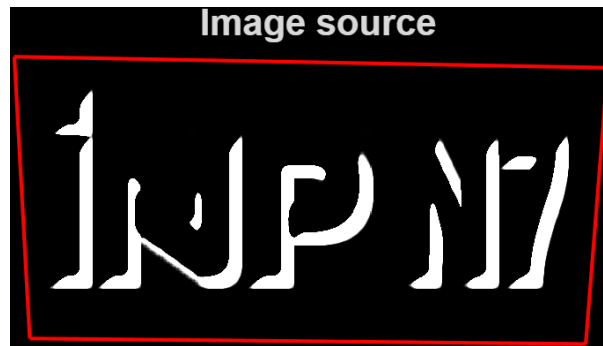


FIGURE 11 – Image du logo INP N7 après le traitement

3.2.3 Résultat final de l'effet de gravure

Finalement en plus d'ajouter du relief, il faut aussi assombrir l'intérieur de la gravure (occlusion ambiante) pour cela on soustrait à notre image source (après le traitement relief et flou) un certain pourcentage de l'image source de base. Appliquer ce masque inversé permet d'assombrir le creux de la gravure et d'obtenir le résultat suivant ;



FIGURE 12 – Exemple de l'effet gravure

3.3 Conclusion

Bien que non hyperréaliste, cet effet développé sur la base de l'équation de Poisson montre à quel point cet outil est efficace pour fusionner des images entre elles sans faire apparaître de coutures. Dans l'objectif d'améliorer ma fonction de rapièçage de la première partie je me suis donc demandé si il était possible d'utiliser les gradients de cette manière pour de l'inpainting par rapièçage.

4 Experimentation Mix TP5 et TP7

Cette section a donc pour but d'expérimenter avec la fonction de rapièçage pour essayer de coller les patches de manière douce à l'aide des gradients de l'image.

4.1 Concept et limite de l'approche locale

Ma première idée a été comme pour le TP7 de recopier les gradients du patch au lieu de recopier sa couleur. Malheureusement comme le montre le résultat ci-dessous le résultat n'est pas probant car les conditions aux limites imposent aux patches de prendre la couleur du fond (noir) sur les parties où la frontière n'est pas encore reconstruite ce qui entraîne évidemment un noircissement quasi-total de la zone à combler :

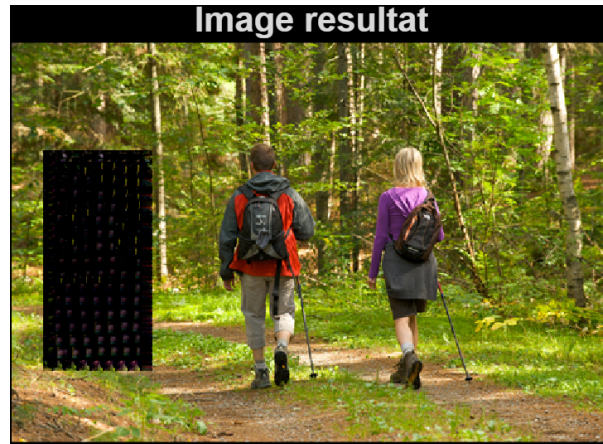


FIGURE 13 – Echec de la méthode locale

Il aurait peut-être fallu pour chaque patch modifier manuellement les conditions aux frontières pour que le patch prenne la couleur de la frontière aux endroits où celle-ci a été reconstruite et garde sa couleur sinon. La mise en oeuvre étant complexe cette solution n'a pas été approfondie.

4.2 Concept et limite de l'approche en deux passes

Une autre solution plus simple mais moins pertinente dans le cas d'une grande zone est restaurer l'image en deux temps. Une fois la région entièrement reconstruite par rapièçage classique, lisser la région obtenue en copiant les gradients de l'image source dans cette région. Cette solution assure à l'image reconstruite d'avoir les mêmes variations d'intensité que l'image source. Cette solution peut sembler intéressante mais perd son sens dans le cas de grandes zones où les possibilités de reconstructions sont multiples (si par exemple l'algorithme a reconstruit un arbre là où il y avait de l'herbe auparavant).



FIGURE 14 – Résultat avant lissage global



FIGURE 15 – Résultat après lissage global

On constate expérimentalement que la prise en compte des gradients n'a pas un impact visuel flagrant, de manière générale l'image est légèrement lissée ce qui rend les coutures un peu moins visibles à certains endroits.

5 TP10 – Experimentations avec des Effets Sonores

5.1 Contexte du TP

Le TP 10 présente dans l'exercice 2 plusieurs manières de modifier un signal audio à l'aide de sa représentation temps-fréquence obtenue par Transformé de Fourier (TFTC). En effet la représentation temps-fréquence permet de réaliser des effets audio de manière très simple tels que des passe-haut ou des passe-bas en tronquant toutes les fréquences d'un côté d'un seuil. Cette section vise à implémenter plusieurs effets audio souvent utilisés dans le cadre de la création musicale et manipulant la représentation temps-fréquence.

Les résultats de ces effets sont écoutables dans le fichier /resultats/TP10 du dépôt Github, le sample original sur lequel ces effets ont été appliqué provient du son Omoide no screen de Yasuko Kawashima (1976).

5.2 Spectral Gate

Le Spectral Gate est un effet assez simple très utilisé dans certains styles de musiques et qui est finalement assez similaire à la méthode de compression de l'exercice 4. En effet le principe de cet effet est de ne conserver que les fréquences supérieures à un certains seuil en décibels fixé par l'utilisateur.

Cet effet permet entre autres de ne conserver que les fréquences fondamentales du son et peut servir pour rajouter un effet "sous l'eau" à la mélodie. Pour permettre plus de flexibilité quand à l'utilisation de cet effet, on rajoute un paramètre mix entre 0 et 1 qui décide des proportions de signal conservés entre le signal fort (au dessus de seuil) et le bruit faible (en dessous). En fixant ce paramètre à 0 on peut conserver uniquement le bruit/la texture du son, le fixer entre 0 et 1 permet de conserver une certaine quantité de détails.

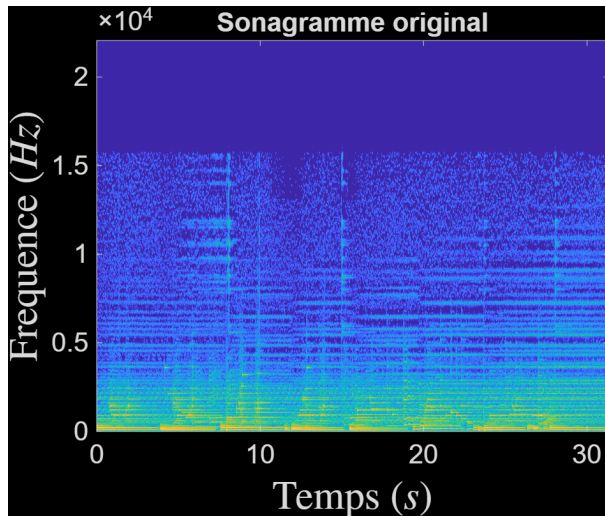


FIGURE 16 – Sonagramme du sample original (sample.mp3)

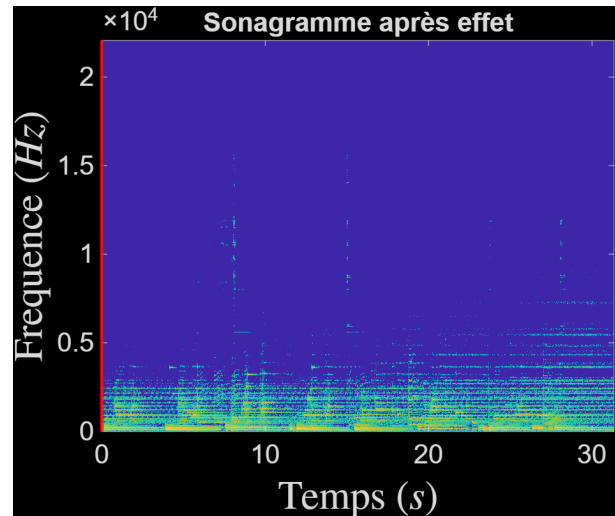


FIGURE 17 – Sonagramme après le spectral gate (son_spectral_gate.mp3)

Ici le spectral gate est appliqué avec un seuil assez élevé et un paramètre mix de 1 ce qui fait que seules les fréquences les plus basses, qui sont généralement les plus fortes, sont conservés à l'exception des sons de la caisse claire qui sont eux aussi toujours reconnaissables bien que moins puissants.

5.3 Modification de la phase d'un son

En plus de pouvoir librement modifier les fréquences, la manipulation du sonagramme d'un signal audio permet aussi de modifier les phases du signal, qui définissent entre autre le timbre du son ainsi que sa structure temporelle locale (transients). Il existe beaucoup de manière de modifier ces phases pour des effets

intéressants (modifications du pitchs, phaser...). Un effet tout simple à réaliser et rendre la phase aléatoire en tirant pour chaque fréquence un nombre entre 0 et 2π . A l'écoute, le son semble plus bruité, flou et moins percutant, cela est dû au déphasage des fréquences entre elles qui fait que par exemple le son de la caisse claire n'arrive pas d'un coup mais étalé sur une petite période de temps. Meme si le son est différent, le sonagramme observé est le même car la phase n'est pas représentée graphiquement sur le sonagramme.

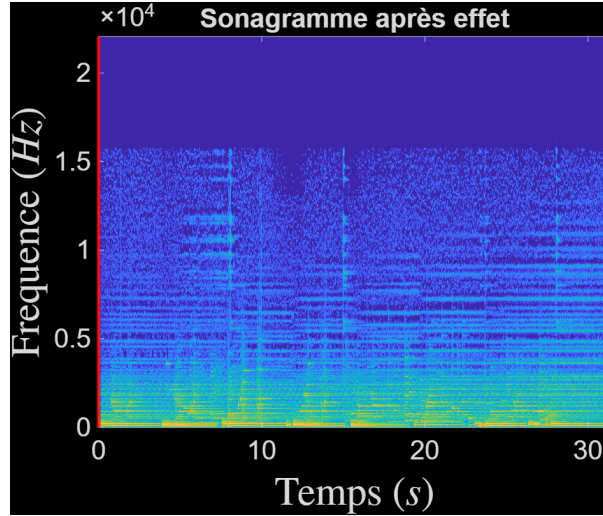


FIGURE 18 – Sonagramme du son avec la phase aléatoire (son_phase_random.mp3)

5.4 Effet de reverb

La reverb, ou en français la réverbération, est un effet extrêmement populaire et largement utilisé dans la production musicale. Son but est d'ajouter de l'espace et de simuler la réflexion du son sur les parois pour donner l'impression qu'il soit joué dans un espace réel et de manière naturelle (chambre, hall, cathédrale...).

5.4.1 Première implémentation

Une implémentation simple de la reverb consiste à l'instant t à jouer $Y(t) + a * Y(t - 1)$ où $Y(t)$ est la colonne à l'instant t du sonagramme du signal, et a est un paramètre entre 0 et 1 appelé feedback qui influe sur la quantité de reverb ajouté au son (proportion du son de $t-1$ qui sera joué à t).

Pour plus de réalisme, on sait que les hautes fréquences se dissipent plus vite dans l'air que les basses du fait de leur plus courte longueur d'onde, on peut donc transformer le scalaire a en un vecteur d'absorption $a * linspace(1, 0.3, nb_freqs)$ qui permet de diminuer plus rapidement les hautes fréquences.

On ajoute à cela une normalisation par fréquence sans laquelle les volumes de certaines fréquences pourraient augmenter fortement. La fonction obtenue est spectral_reverb.m .

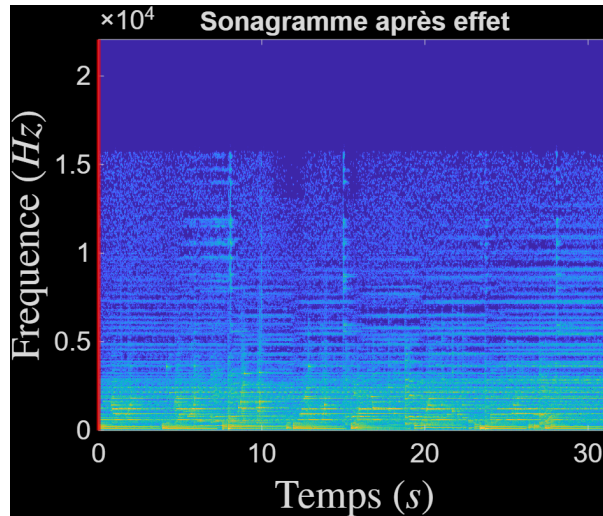


FIGURE 19 – Sonagramme du son avec la reverb 1 (son_reverb1.mp3)

Cette première implémentation de la reverb n'est pas la plus agréable à l'oreil, en effet le son sonne "métallique" et ne semble pas être projeté dans un grand espace comme voulu.

5.4.2 Amélioration avec pre-delay et phase

En effet, jouer le son $t-1$ à l'instant t revient à dire que le son a "rebondi" contre la paroi et est revenu en un seul Δt , ce qui revient à simuler un espace très confiné. On ajoute donc dans la fonction *spectral_reverb2.m* un paramètre pre-delay qui simule le temps que met un son pour rebondir contre la paroi et nous reparvenir. A l'instant t on joue donc $Y(t) + a * Y(t - predelay)$.

En plus de ça, l'aspect métallique vient aussi du fait que la phase reste exactement identique entre le son avant et après le rebond contre la paroi. Physiquement parlant, les irrégularités du mur créent un phénomène de diffusion qui ajoute un bruit aléatoire à la phase.

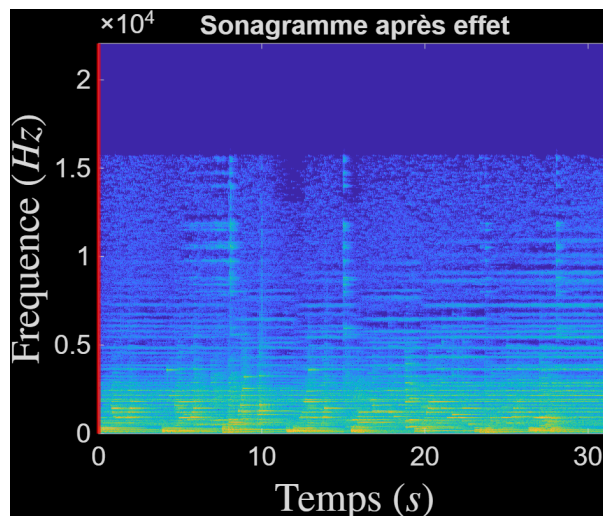


FIGURE 20 – Sonagramme du son avec la reverb 2 (son_reverb2.mp3)

5.4.3 Amélioration avec flou temporel

En écoutant le son de la 2e version de la reverb, on se rend compte que fixer un trop grand pre-delay sonne comme si chaque son se répétait nettement plusieurs fois de manière rythmique, l'oreille arrive à séparer le moment où le son part à celui où le son revient. En réalité, le son ne revient pas après son rebond à un seul instant t précis, au contraire, certaines ondes vont rebondir contre différents endroits du mur ou même contre plusieurs mur avant de revenir à des instants légèrement décalés. On peut donc essayer d'étaler le rebond sur plusieurs instants en jouant à l'instant t : $Y(t) + a * (Y(t - \text{predelay}) + Y(t - \text{predelay} - 1) + Y(t - \text{predelay} - 2)) / 3$. Cette moyenne glissante sur les 3 frames passées permet d'obtenir un espace acoustique large, dense et plus naturel comme voulu initialement.

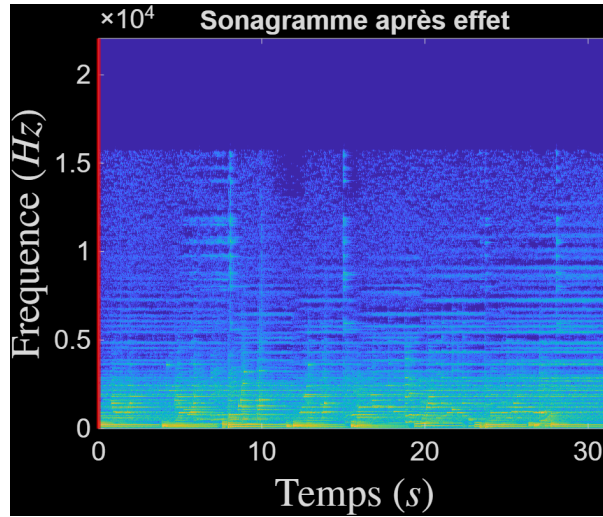


FIGURE 21 – Sonagramme du son avec la reverb finale (son_reverb3.mp3)

5.5 Conclusion sur les effets sonores

En conclusion, la manipulation de signaux audios dans le domaine fréquentiel est très largement utilisée dans le milieu musical. Il existe de nombreux pluggins (logiciels) qui implémentent de cette manière différents effets tout les plus originaux les uns que les autres. Le plugin SpecOps développé par Unfiltered Audio est par exemple un plugin très poussé dans ce sens là, il permet d'appliquer 3 effets différents sur 3 plages de fréquences différentes, mais l'utilisateur peut aussi ajuster directement des paramètres techniques tels que la taille de la FFT (paramètre FFT Size à gauche) ou le type de fenêtre utilisé pour la FFT (je suis très fan de ce plugin !).



FIGURE 22 – Interface du plugin SpecOps

6 TP12 – Séparation de Sources

6.1 Contexte du TP

Le TP12 présente différentes techniques permettant de décomposer un signal audio en ses composantes individuelles, telles que l'extraction de la voix, l'isolation de la batterie ou la séparation des pistes instrumentales. Ces procédés peuvent être utiles dans différents contextes, que ce soit pour la production musicale ou bien pour du traitement du signal pour faire de la traduction par exemple. Cependant les méthodes des exercices 1 et 2 du TP12 (séparation harmonique/percussive et NMF) ne sont pas entièrement satisfaisantes et présentent vite leur limite selon les sons utilisés, on obtient rarement exactement le son voulu.

Aujourd'hui la solution la plus efficace pour ce genre de tâches complexes est de faire du machine learning. En entraînant un réseau de neurones sur une base de données suffisamment grande, on peut arriver à des résultats très satisfaisants comme le montre Spleeter, le réseau entraîné par Deezer.

6.2 Apprentissage d'un réseau de type U-Net

Pour aborder le problème complexe de la séparation de sources, nous avons implémenté une architecture neuronale de type U-Net. Son architecture s'adapte parfaitement au traitement des spectrogrammes. En sortie, le réseau ne génère pas directement de l'audio, mais apprend à prédire un masque spectral qui sera ensuite appliqué au signal d'entrée.

L'architecture se divise en trois grandes parties :

- L'encodeur : composé de plusieurs blocs comprenant une couche de Conv2D avec un stride de 2 pour réduire la taille, un layer de BatchNormalization pour stabiliser l'apprentissage et une activation leakyReLU. L'objectif de cette phase est de compresser l'information spatiale et fréquentielle pour en extraire des caractéristiques de haut niveau comme le timbre, le rythme, etc. Il s'agit du feature extractor.
- Le bottleneck : C'est le point le plus profond du réseau où la résolution fréquentielle est minimale. A partir de ce stade on augmente la résolution pour générer un masque précis.
- Le décodeur : le décodeur effectue le chemin inverse via des blocs de déconvolution. On utilise des skips pour relier les activations de l'encodeur aux sorties des layers du décodeur de même résolutions afin de palier à la perte d'informations spatiales et des détails due à la compression.
- Output : La couche de sortie est constituée de deux filtres, ici un pour le chant et un pour l'accompagnement.

6.3 Optimisations du modèle

Bien que fonctionnel, le modèle de base présentait un léger surapprentissage et stagnait au bout d'un certain nombre d'époques, plusieurs stratégies ont été mises en place pour essayer d'améliorer les performances du modèle.

- Learning rate : Une première stratégie a été de modifier le learning rate dynamiquement à l'aide d'un callback *ReduceLROnPlateau*. Dès que la loss du modèle ne diminue plus pendant un certain nombre d'époque, le learning rate est diminué d'un certain ratio. Cet ajustement dynamique permet d'éviter d'avoir un trop petit learning rate (apprentissage lent) ou bien un trop grand (apprentissage instable).

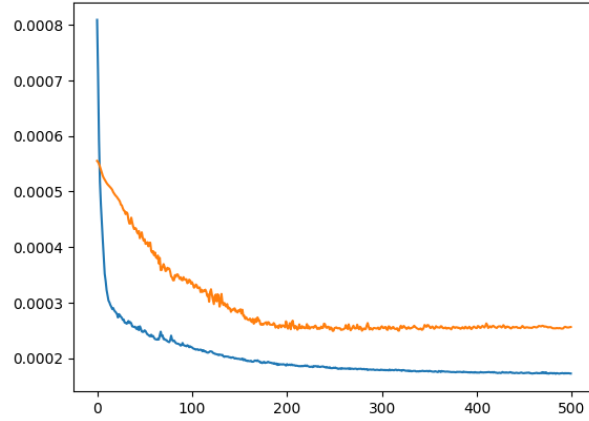


FIGURE 23 – Courbe d'apprentissage 1

- Robustesse : une stratégie supplémentaire est d'augmenter la robustesse de notre réseau pour éviter l'overfitting, pour cela on peut rajouter des layers de Dropout comme recommandé dans le sujet. Un layer de Dropout tronque aléatoirement une certaine quantité de poids en les mettant à 0 afin de forcer le réseau à ne pas dépendre de quelques poids qui prendraient trop d'importance. En plus du Dropout, on peut aussi faire de la Data Augmentation pour forcer le réseau à généraliser et éviter qu'il n'apprenne les pistes d'entraînement par cœur. Pour cela on ajoute en entrée un bruit gaussien sur notre son. Cette stratégie a permis de réduire légèrement l'écart entre la perte d'entraînement et de validation qui descend jusqu'à environ 2.24×10^{-4} .

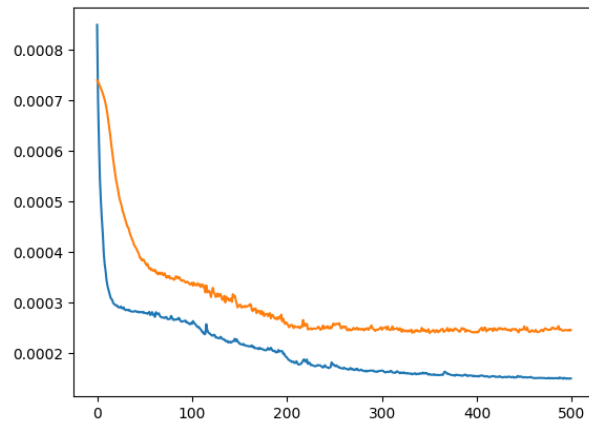


FIGURE 24 – Courbe d'apprentissage 1

Les résultats de notre réseau sont convaincants mais restent loin des performances affichées par des modèles tels que celui de Deezer. Pour de meilleurs résultats, il faudrait certainement augmenter la taille du réseau et des filtres de convolution pour entraîner plus de poids ainsi qu'avoir une base de données plus grande pour mieux généraliser l'apprentissage.

7 Conclusion Générale

D'une part, l'étude de méthodes appliquées à l'image (TP5 et TP7) nous a montré comment des outils mathématiques, comme la copie de patches ou l'équation de Poisson, permettent de tromper l'œil humain pour effacer ou incruster des éléments de manière efficace. D'autre part, le traitement du signal audio (TP10 et TP12) a mis en évidence l'utilité d'une manipulation manuelle des fréquences via la Transformée de Fourier et a également montré comment l'utilisation du Deep Learning permet aujourd'hui de résoudre des tâches plus complexes telles que la séparation de sources.

En conclusion, ce projet a été une bonne opportunité de mettre en pratique l'évolution des techniques de manipulation de l'image et du son, en allant des méthodes classiques vers l'IA.